If you compare this with the earlier schema file, you can see the pattern of the JSON representation of the schema.

The AVP `put-schema` API expects this JSON object to be the value of another JSON and stored in the `cedarJson` field as an escaped string. To do this, we use `jq` to convert the JSON to a string by escaping all embedded quotation marks with a backslash and combining all lines into a single line with no line breaks:

```
jq -Rs '{cedarJson: .}' acme.cedarschema.json > schema-definition.j
son
```

Once we have the schema as a JSON object, we can upload the schema to our policy store using the following command:

```
aws verifiedpermissions put-schema \
  --policy-store-id "$STORE_ID" \
  --definition file://schema-definition.json
```

If everything is correct, you'll receive a JSON response showing the policy store ID, the namespaces within it, and the creation and update dates. The namespace should match the one in the schema you just uploaded to the policy store. In our case, it's `ACME`.

A.4 Uploading policies to AVP

With the schema in place, ACME Corp can now upload Cedar policies that define how access decisions are made. In AVP, each policy is created using the `create-policy` command. AVP validates each policy against the schema before storing it, ensuring that errors are caught early.

We'll begin with the core "view" policies and then add edit, share, and global guardrail rules. Consider this as gradually building the access model, layer by layer.